

Lab 10 — Source Code

InvoiceDoc2 Project

database/sql/005_invoice_lab4_delta.sql

```
\set ON_ERROR_STOP on

-- Receipt header table
CREATE TABLE IF NOT EXISTS receipt (
  id          bigint primary key,
  created_at  timestampz not null default now(),
  receipt_no  text unique not null,
  receipt_date date not null,
  customer_id bigint not null references customer(id),
  payment_method text not null default 'cash',
  payment_notes text,
  total_received numeric(14,2) not null default 0.00
);

-- Receipt line item table
CREATE TABLE IF NOT EXISTS receipt_line_item (
  id          bigint primary key,
  created_at  timestampz not null default now(),
  receipt_id  bigint not null references receipt(id) on delete cascade,
  invoice_id  bigint not null references invoice(id),
  amount_received numeric(14,2) not null default 0.00
);

-- View: invoice_received_view
CREATE OR REPLACE VIEW invoice_received_view AS
SELECT
  c.id          AS customer_id,
  i.id          AS invoice_id,
  i.invoice_no,
  i.amount_due,
  COALESCE(SUM(rli.amount_received), 0) AS amount_received,
  i.amount_due - COALESCE(SUM(rli.amount_received), 0) AS amount_remain
FROM invoice i
JOIN customer c ON c.id = i.customer_id
LEFT JOIN receipt_line_item rli ON rli.invoice_id = i.id
GROUP BY c.id, i.id, i.invoice_no, i.amount_due;
```

server/src/services/receipts.service.js

```
import { pool } from "../db/pool.js";

export async function listReceipts({
  search = "",
  page = 1,
  limit = 10,
  sortBy = "receipt_date",
  sortDir = "desc",
} = {}) {
  const offset = (Number(page) - 1) * Number(limit);

  const allowedSort = ["receipt_no", "receipt_date", "customer_code", "customer_name", "payment_method", "total_received"];
  const sortColumn = allowedSort.includes(sortBy) ? sortBy : "receipt_date";
  const sortDirection = sortDir === "asc" ? "ASC" : "DESC";

  const searchParam = `%${search}%`;

  const countResult = await pool.query(
    `SELECT COUNT(*) as total
     FROM receipt r
     JOIN customer c ON c.id = r.customer_id
     WHERE r.receipt_no ILIKE $1 OR c.name ILIKE $1`,
    [searchParam],
  );
  const total = Number(countResult.rows[0].total);

  const { rows } = await pool.query(
    `SELECT r.receipt_no, r.receipt_date, c.code as customer_code, c.name as customer_name,
     r.payment_method, r.total_received`
  );
}
```

```

    FROM receipt r
    JOIN customer c ON c.id = r.customer_id
    WHERE r.receipt_no ILIKE $1 OR c.name ILIKE $1
    ORDER BY ${sortColumn} ${sortDirection} NULLS LAST, r.id DESC
    LIMIT $2 OFFSET $3`,
    [searchParam, Number(limit), offset],
  );

  return {
    data: rows,
    total,
    page: Number(page),
    limit: Number(limit),
    totalPages: Math.ceil(total / Number(limit)),
  };
}

export async function getReceipt(receiptNo) {
  const headerRes = await pool.query(
    `SELECT r.id, r.receipt_no, r.receipt_date, r.payment_method, r.payment_notes, r.total_received,
      c.code as customer_code, c.name as customer_name
    FROM receipt r
    JOIN customer c ON c.id = r.customer_id
    WHERE r.receipt_no = $1`,
    [receiptNo],
  );
  if (headerRes.rowCount === 0) return null;

  const header = headerRes.rows[0];
  const receiptId = header.id;

  // amount_already_received excludes current receipt's own payments
  const linesRes = await pool.query(
    `SELECT rli.id, i.invoice_no, i.amount_due,
      (SELECT COALESCE(SUM(rli2.amount_received), 0)
       FROM receipt_line_item rli2
       WHERE rli2.invoice_id = rli.invoice_id AND rli2.receipt_id != $2) AS amount_already_received,
      rli.amount_received
    FROM receipt_line_item rli
    JOIN invoice i ON i.id = rli.invoice_id
    WHERE rli.receipt_id = $1
    ORDER BY rli.id`,
    [receiptId, receiptId],
  );

  return { header, line_items: linesRes.rows };
}

export async function generateReceiptNo(client) {
  const yy = new Date().getFullYear().toString().slice(-2);
  const res = await client.query("SELECT MAX(id) as m FROM receipt");
  const nextId = (Number(res.rows[0].m) || 0) + 1;
  return `RCT${yy}-${nextId.toString().padStart(5, "0")}`;
}

export async function createReceipt({ receipt_no, receipt_date, customer_code, payment_method, payment_notes, line_items }) {
  const client = await pool.connect();
  try {
    await client.query("begin");

    const custRes = await client.query("SELECT id FROM customer WHERE code = $1", [customer_code]);
    if (custRes.rowCount === 0) throw new Error(`Customer not found: ${customer_code}`);
    const customer_id = custRes.rows[0].id;

    let resolvedReceiptNo = receipt_no;
    if (!resolvedReceiptNo || String(resolvedReceiptNo).trim() === "") {
      resolvedReceiptNo = await generateReceiptNo(client);
    }

    const total_received = line_items.reduce((s, li) => s + Number(li.amount_received), 0);

    const rcptRes = await client.query(
      `INSERT INTO receipt (id, created_at, receipt_no, receipt_date, customer_id, payment_method, payment_notes, total_received)
      VALUES ((SELECT COALESCE(MAX(id),0)+1 FROM receipt), now(), $1, $2, $3, $4, $5, $6)
      RETURNING id, receipt_no`,
      [resolvedReceiptNo, receipt_date, customer_id, payment_method || "cash", payment_notes || null, total_received],
    );

    const receipt_id = rcptRes.rows[0].id;

    for (const li of line_items) {
      const invRes = await client.query("SELECT id FROM invoice WHERE invoice_no = $1", [li.invoice_no]);
    }
  }
}

```

```

    if (invRes.rowCount === 0) throw new Error(`Invoice not found: ${li.invoice_no}`);
    const invoice_id = invRes.rows[0].id;

    await client.query(
      `INSERT INTO receipt_line_item (id, created_at, receipt_id, invoice_id, amount_received)
      VALUES ((SELECT COALESCE(MAX(id),0)+1 FROM receipt_line_item), now(), $1, $2, $3)`,
      [receipt_id, invoice_id, Number(li.amount_received)],
    );
  }

  await client.query("commit");
  return { receipt_no: rcptRes.rows[0].receipt_no };
} catch (err) {
  await client.query("rollback");
  throw err;
} finally {
  client.release();
}
}

export async function updateReceipt(receiptNo, { receipt_date, customer_code, payment_method, payment_notes, line_items }) {
  const client = await pool.connect();
  try {
    await client.query("begin");

    const existing = await client.query("SELECT id FROM receipt WHERE receipt_no = $1", [receiptNo]);
    if (existing.rowCount === 0) return null;
    const receipt_id = existing.rows[0].id;

    const custRes = await client.query("SELECT id FROM customer WHERE code = $1", [customer_code]);
    if (custRes.rowCount === 0) throw new Error(`Customer not found: ${customer_code}`);
    const customer_id = custRes.rows[0].id;

    const total_received = line_items.reduce((s, li) => s + Number(li.amount_received), 0);

    await client.query(
      `UPDATE receipt SET receipt_date=$1, customer_id=$2, payment_method=$3, payment_notes=$4, total_received=$5
      WHERE id=$6`,
      [receipt_date, customer_id, payment_method || "cash", payment_notes || null, total_received, receipt_id],
    );

    // DELETE old line items then re-INSERT
    await client.query("DELETE FROM receipt_line_item WHERE receipt_id = $1", [receipt_id]);

    for (const li of line_items) {
      const invRes = await client.query("SELECT id FROM invoice WHERE invoice_no = $1", [li.invoice_no]);
      if (invRes.rowCount === 0) throw new Error(`Invoice not found: ${li.invoice_no}`);
      const invoice_id = invRes.rows[0].id;

      await client.query(
        `INSERT INTO receipt_line_item (id, created_at, receipt_id, invoice_id, amount_received)
        VALUES ((SELECT COALESCE(MAX(id),0)+1 FROM receipt_line_item), now(), $1, $2, $3)`,
        [receipt_id, invoice_id, Number(li.amount_received)],
      );
    }

    await client.query("commit");
    return { receipt_no: receiptNo };
  } catch (err) {
    await client.query("rollback");
    throw err;
  } finally {
    client.release();
  }
}

export async function deleteReceipt(receiptNo) {
  const res = await pool.query("DELETE FROM receipt WHERE receipt_no = $1 RETURNING id", [receiptNo]);
  if (res.rowCount === 0) return null;
  return { ok: true };
}

export async function listUnpaidInvoicesByCustomer(customerCode, excludeReceiptNo = null) {
  const custRes = await pool.query("SELECT id FROM customer WHERE code = $1", [customerCode]);
  if (custRes.rowCount === 0) return [];
  const customer_id = custRes.rows[0].id;

  let excludeReceiptId = null;
  if (excludeReceiptNo) {
    const rcptRes = await pool.query("SELECT id FROM receipt WHERE receipt_no = $1", [excludeReceiptNo]);
    if (rcptRes.rowCount > 0) excludeReceiptId = rcptRes.rows[0].id;
  }
}

```

```

const { rows } = await pool.query(
  `SELECT i.invoice_no, i.invoice_date, i.amount_due,
    COALESCE(SUM(CASE WHEN ($2::bigint IS NULL OR rli.receipt_id != $2) THEN rli.amount_received ELSE 0 END), 0) AS amount,
    i.amount_due - COALESCE(SUM(CASE WHEN ($2::bigint IS NULL OR rli.receipt_id != $2) THEN rli.amount_received ELSE 0 END), 0) AS amount_received
  FROM invoice i
  LEFT JOIN receipt_line_item rli ON rli.invoice_id = i.id
  WHERE i.customer_id = $1
  GROUP BY i.id, i.invoice_no, i.invoice_date, i.amount_due
  HAVING i.amount_due - COALESCE(SUM(CASE WHEN ($2::bigint IS NULL OR rli.receipt_id != $2) THEN rli.amount_received ELSE 0 END), 0) > 0
  ORDER BY i.invoice_date`,
  [customer_id, excludeReceiptId],
);

return rows;
}

```

server/src/controllers/receipts.controller.js

```

import * as receiptsService from "../services/receipts.service.js";
import { sendList, sendOne, sendCreated, sendOk, sendData, sendError } from "../utils/response.js";
import logger from "../utils/logger.js";

export async function handleList(req, res) {
  try {
    const result = await receiptsService.listReceipts(req.query);
    sendList(res, result);
  } catch (err) {
    logger.error("listReceipts failed", { error: err?.message ?? String(err) });
    sendError(res, err?.message ?? String(err), 500);
  }
}

export async function handleGet(req, res) {
  try {
    const receiptNo = decodeURIComponent(req.params.receiptNo || "");
    const result = await receiptsService.getReceipt(receiptNo);
    if (!result) return sendError(res, "Receipt not found", 404);
    sendOne(res, result);
  } catch (err) {
    logger.error("getReceipt failed", { receiptNo: req.params.receiptNo, error: err?.message ?? String(err) });
    sendError(res, err?.message ?? String(err), 500);
  }
}

export async function handleCreate(req, res) {
  try {
    const result = await receiptsService.createReceipt(req.body);
    sendCreated(res, result);
  } catch (err) {
    logger.error("createReceipt failed", { error: err?.message ?? String(err) });
    sendError(res, err?.message ?? String(err), 500);
  }
}

export async function handleUpdate(req, res) {
  try {
    const receiptNo = decodeURIComponent(req.params.receiptNo || "");
    const result = await receiptsService.updateReceipt(receiptNo, req.body);
    if (!result) return sendError(res, "Receipt not found", 404);
    sendOk(res, result);
  } catch (err) {
    logger.error("updateReceipt failed", { receiptNo: req.params.receiptNo, error: err?.message ?? String(err) });
    sendError(res, err?.message ?? String(err), 500);
  }
}

export async function handleDelete(req, res) {
  try {
    const receiptNo = decodeURIComponent(req.params.receiptNo || "");
    const result = await receiptsService.deleteReceipt(receiptNo);
    if (!result) return sendError(res, "Receipt not found", 404);
    sendOk(res, result);
  } catch (err) {
    logger.error("deleteReceipt failed", { receiptNo: req.params.receiptNo, error: err?.message ?? String(err) });
    sendError(res, err?.message ?? String(err), 500);
  }
}

export async function handleListUnpaidInvoices(req, res) {

```

```

try {
  const { customer_code, receipt_no } = req.query;
  if (!customer_code) return sendError(res, "customer_code is required", 400);
  const result = await receiptsService.listUnpaidInvoicesByCustomer(customer_code, receipt_no || null);
  sendData(res, result);
} catch (err) {
  logger.error("listUnpaidInvoices failed", { error: err?.message ?? String(err) });
  sendError(res, err?.message ?? String(err), 500);
}
}

```

server/src/routes/receipts.routes.js

```

import { Router } from "express";
import * as c from "../controllers/receipts.controller.js";

const r = Router();
r.get("/", c.handleList);
r.get("/unpaid-invoices", c.handleListUnpaidInvoices);
r.get("/:receiptNo", c.handleGet);
r.post("/", c.handleCreate);
r.put("/:receiptNo", c.handleUpdate);
r.delete("/:receiptNo", c.handleDelete);

export default r;

```

server/src/services/receiptReports.service.js

```

import { pool } from "../db/pool.js";

export async function getReceiptList({ date_from, date_to, customer_code, page = 1, limit = 10 } = {}) {
  const conditions = [];
  const params = [];
  let idx = 1;

  if (date_from) { conditions.push(`r.receipt_date >= ${idx++}`); params.push(date_from); }
  if (date_to) { conditions.push(`r.receipt_date <= ${idx++}`); params.push(date_to); }
  if (customer_code) { conditions.push(`c.code = ${idx++}`); params.push(customer_code); }

  const where = conditions.length > 0 ? `WHERE ${conditions.join(" AND ")}` : "";
  const offset = (Number(page) - 1) * Number(limit);

  const countRes = await pool.query(
    `SELECT COUNT(*) as total
     FROM receipt r
     JOIN customer c ON c.id = r.customer_id
     ${where}`,
    params,
  );
  const total = Number(countRes.rows[0].total);

  const { rows } = await pool.query(
    `SELECT r.receipt_no, r.receipt_date, c.code as customer_code, c.name as customer_name,
     r.payment_method, r.payment_notes, r.total_received
     FROM receipt r
     JOIN customer c ON c.id = r.customer_id
     ${where}
     ORDER BY r.receipt_date DESC, r.id DESC
     LIMIT ${idx++} OFFSET ${idx++}`,
    [...params, Number(limit), offset],
  );

  return {
    data: rows,
    total,
    page: Number(page),
    limit: Number(limit),
    totalPages: Math.ceil(total / Number(limit)),
  };
}

export async function getInvoiceReceiptReport({ date_from, date_to, customer_code, page = 1, limit = 10 } = {}) {
  const conditions = [];
  const params = [];
  let idx = 1;

```

```

if (date_from) { conditions.push(`i.invoice_date >= ${idx++}`); params.push(date_from); }
if (date_to) { conditions.push(`i.invoice_date <= ${idx++}`); params.push(date_to); }
if (customer_code) { conditions.push(`c.code = ${idx++}`); params.push(customer_code); }

const where = conditions.length > 0 ? `WHERE ${conditions.join(" AND ")}` : "";
const offset = (Number(page) - 1) * Number(limit);

const countRes = await pool.query(
  `WITH total_received_cte AS (
    SELECT invoice_id, COALESCE(SUM(amount_received), 0) AS total_received
    FROM receipt_line_item
    GROUP BY invoice_id
  )
  SELECT COUNT(*) as total
  FROM invoice i
  JOIN customer c ON c.id = i.customer_id
  LEFT JOIN receipt_line_item rli ON rli.invoice_id = i.id
  LEFT JOIN receipt r ON r.id = rli.receipt_id
  LEFT JOIN total_received_cte trc ON trc.invoice_id = i.id
  ${where}`,
  params,
);
const total = Number(countRes.rows[0].total);

const { rows } = await pool.query(
  `WITH total_received_cte AS (
    SELECT invoice_id, COALESCE(SUM(amount_received), 0) AS total_received
    FROM receipt_line_item
    GROUP BY invoice_id
  )
  SELECT i.invoice_no, i.invoice_date, c.code as customer_code, c.name as customer_name,
    i.amount_due,
    COALESCE(trc.total_received, 0) AS total_received,
    i.amount_due - COALESCE(trc.total_received, 0) AS amount_remain,
    r.receipt_no, r.receipt_date AS receipt_date, rli.amount_received AS receipt_amount
  FROM invoice i
  JOIN customer c ON c.id = i.customer_id
  LEFT JOIN receipt_line_item rli ON rli.invoice_id = i.id
  LEFT JOIN receipt r ON r.id = rli.receipt_id
  LEFT JOIN total_received_cte trc ON trc.invoice_id = i.id
  ${where}
  ORDER BY i.invoice_date DESC, i.id DESC, r.receipt_date
  LIMIT ${idx++} OFFSET ${idx++}`,
  [...params, Number(limit), offset],
);

return {
  data: rows,
  total,
  page: Number(page),
  limit: Number(limit),
  totalPages: Math.ceil(total / Number(limit)),
};
}

```

server/src/controllers/receiptReports.controller.js

```

import * as receiptReportsService from "../../services/receiptReports.service.js";
import { sendList, sendError } from "../../utils/response.js";
import logger from "../../utils/logger.js";

export async function handleReceiptList(req, res) {
  try {
    const result = await receiptReportsService.getReceiptList(req.query);
    sendList(res, result);
  } catch (err) {
    logger.error("getReceiptList failed", { error: err?.message ?? String(err) });
    sendError(res, err?.message ?? String(err), 500);
  }
}

export async function handleInvoiceReceiptReport(req, res) {
  try {
    const result = await receiptReportsService.getInvoiceReceiptReport(req.query);
    sendList(res, result);
  } catch (err) {
    logger.error("getInvoiceReceiptReport failed", { error: err?.message ?? String(err) });
    sendError(res, err?.message ?? String(err), 500);
  }
}

```

```
}
```

server/src/routes/receiptReports.routes.js

```
import { Router } from "express";
import * as c from "../controllers/receiptReports.controller.js";

const r = Router();
r.get("/receipt-list", c.handleReceiptList);
r.get("/invoice-receipt", c.handleInvoiceReceiptReport);

export default r;
```

client/src/api/receipts.api.js

```
import { http } from "../http.js";

function unwrap(res) {
  if (res && res.success === false && res.error) throw new Error(res.error.message);
  return res;
}

export async function listReceipts(params = {}) {
  const query = new URLSearchParams(params).toString();
  const res = unwrap(await http(`/api/receipts${query ? `?` + query : ""}`));
  return { data: res.data, ...(res.meta || {}) };
}

export async function getReceipt(receiptNo) {
  const res = unwrap(await http(`/api/receipts/${encodeURIComponent(receiptNo)}`));
  return res.data;
}

export async function createReceipt(body) {
  const res = unwrap(await http("/api/receipts", { method: "POST", body: JSON.stringify(body) }));
  return res.data;
}

export async function updateReceipt(receiptNo, body) {
  const res = unwrap(await http(`/api/receipts/${encodeURIComponent(receiptNo)}`, { method: "PUT", body: JSON.stringify(body) }));
  return res.data;
}

export async function deleteReceipt(receiptNo) {
  const res = unwrap(await http(`/api/receipts/${encodeURIComponent(receiptNo)}`, { method: "DELETE" }));
  return res.data;
}

export async function listUnpaidInvoices(customerCode, receiptNo = null) {
  const params = new URLSearchParams({ customer_code: customerCode });
  if (receiptNo) params.append("receipt_no", receiptNo);
  const res = unwrap(await http(`/api/receipts/unpaid-invoices?${params.toString()}`));
  return res.data;
}
```

client/src/api/receiptReports.api.js

```
import { http } from "../http.js";

function unwrap(res) {
  if (res && res.success === false && res.error) throw new Error(res.error.message);
  return res;
}

function buildQuery(params) {
  const filtered = Object.fromEntries(
    Object.entries(params).filter(([v]) => v !== null && v !== undefined && v !== "")
  );
  return new URLSearchParams(filtered).toString();
}

export async function fetchReceiptList(params = {}) {
  const query = buildQuery(params);
```

```

const res = unwrap(await http(`/api/receipt-reports/receipt-list${query ? `?${query}` : ""}`));
return { data: res.data, ...(res.meta || {}) };
}

export async function fetchInvoiceReceiptReport(params = {}) {
const query = buildQuery(params);
const res = unwrap(await http(`/api/receipt-reports/invoice-receipt${query ? `?${query}` : ""}`));
return { data: res.data, ...(res.meta || {}) };
}

```

client/src/pages/receipts/ReceiptList.jsx

```

import React from "react";
import { toast } from "react-toastify";
import { listReceipts, deleteReceipt } from "../../../api/receipts.api.js";
import { formatBaht, formatDate } from "../../../utils.js";
import DataList from "../../../components/DataList.jsx";
import { ConfirmModal, AlertModal } from "../../../components/Modal.jsx";

export default function ReceiptList() {
const fetchData = React.useCallback((params) => listReceipts(params), []);
const [confirmModal, setConfirmModal] = React.useState({ isOpen: false, id: null });
const [alertModal, setAlertModal] = React.useState({ isOpen: false, message: "" });
const [refreshTrigger, setRefreshTrigger] = React.useState(0);

const closeConfirm = () => setConfirmModal({ isOpen: false, id: null });

const handleDelete = (id) => {
setConfirmModal({ isOpen: true, id });
};

const confirmDelete = async () => {
try {
await deleteReceipt(confirmModal.id);
closeConfirm();
setRefreshTrigger((t) => t + 1);
toast.success("Receipt deleted.");
} catch (e) {
const msg = String(e.message || e);
toast.error(msg);
setAlertModal({ isOpen: true, message: "Error: " + msg });
closeConfirm();
}
};

const columns = [
{ key: "receipt_no", label: "Receipt No" },
{ key: "receipt_date", label: "Date", render: (v) => formatDate(v) },
{ key: "customer_code", label: "Customer Code" },
{ key: "customer_name", label: "Customer Name" },
{ key: "payment_method", label: "Payment Method" },
{ key: "total_received", label: "Total Received", align: "right", render: (v) => formatBaht(v) },
];

return (
<>
<ConfirmModal
isOpen={confirmModal.isOpen}
onClose={closeConfirm}
onConfirm={confirmDelete}
closeOnConfirm={false}
title="Delete Receipt"
message="Are you sure you want to delete this receipt?"
confirmText="Delete"
/>
<AlertModal
isOpen={alertModal.isOpen}
onClose={() => setAlertModal({ isOpen: false, message: "" })}
title="Error"
message={alertModal.message}
/>
<DataList
refreshTrigger={refreshTrigger}
title="Receipts"
fetchData={fetchData}
columns={columns}
searchPlaceholder="Search receipt no, customer..."
itemName="receipts"
basePath="/receipts"

```



```

        itemKey="receipt_no"
        onDelete={handleDelete}
      />
    </>
  );
}

```

client/src/pages/receipts/InvoicePickerModal.jsx

```

import React from "react";
import { createPortal } from "react-dom";
import { listUnpaidInvoices } from "../../api/receipts.api.js";
import { formatBaht, formatDate } from "../../utils.js";

export default function InvoicePickerModal({ isOpen, onClose, onSelect, customerCode, excludeReceiptNo }) {
  const [invoices, setInvoices] = React.useState([]);
  const [loading, setLoading] = React.useState(false);
  const [error, setError] = React.useState("");

  React.useEffect(() => {
    if (!isOpen || !customerCode) {
      setInvoices([]);
      return;
    }
    setLoading(true);
    setError("");
    listUnpaidInvoices(customerCode, excludeReceiptNo)
      .then((data) => setInvoices(data || []))
      .catch((e) => setError(String(e.message || e)))
      .finally(() => setLoading(false));
  }, [isOpen, customerCode, excludeReceiptNo]);

  if (!isOpen) return null;

  return createPortal(
    <div
      style={{
        position: "fixed",
        inset: 0,
        background: "rgba(0,0,0,0.5)",
        display: "flex",
        alignItems: "center",
        justifyContent: "center",
        zIndex: 10000,
        padding: 24,
      }}
      onClick={onClose}
    >
      <div
        style={{
          background: "white",
          borderRadius: "var(--radius)",
          boxShadow: "var(--shadow-lg)",
          maxWidth: 760,
          width: "100%",
          maxHeight: "80vh",
          overflow: "auto",
        }}
        onClick={(e) => e.stopPropagation()}
      >
        <div style={{ padding: "20px 24px", borderBottom: "1px solid var(--border)" }}>
          <h3 style={{ margin: 0, fontSize: "1.1rem", fontWeight: 600 }}>Select Invoice</h3>
        </div>
        <div style={{ padding: 24 }}>
          {loading && (
            <div style={{ textAlign: "center", padding: 24, color: "var(--text-muted)" }}>Loading...</div>
          )}
          {error && <div className="alert alert-error">{error}</div>}
          {!loading && !error && (
            <div className="table-container">
              <table className="modern-table">
                <thead>
                  <tr>
                    <th>Invoice No</th>
                    <th>Invoice Date</th>
                    <th className="text-right">Amount Due</th>
                    <th className="text-right">Already Received</th>
                    <th className="text-right">Remaining</th>
                    <th>Action</th>

```

```

        </tr>
      </thead>
      <tbody>
        {invoices.length === 0 && (
          <tr>
            <td colSpan={6} style={{ textAlign: "center", padding: 24, color: "var(--text-muted)" }}>
              No unpaid invoices found.
            </td>
          </tr>
        )}
        {invoices.map((inv) => (
          <tr key={inv.invoice_no}>
            <td>{inv.invoice_no}</td>
            <td>{formatDate(inv.invoice_date)}</td>
            <td className="text-right">{formatBaht(inv.amount_due)}</td>
            <td className="text-right">{formatBaht(inv.amount_already_received)}</td>
            <td className="text-right">{formatBaht(inv.amount_remain)}</td>
            <td>
              <button
                className="btn btn-primary"
                style={{ fontSize: "0.75rem", padding: "4px 10px" }}
                onClick={() => onSelect(inv)}
              >
                Select
              </button>
            </td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
</div>
<div style={{ padding: "16px 24px", borderTop: "1px solid var(--border)", display: "flex", justifyContent: "flex-end" }}>
  <button className="btn btn-outline" onClick={onClose}>Close</button>
</div>
</div>
</div>,
document.body,
);
}

```

client/src/pages/receipts/ReceiptPage.jsx

```

import React from "react";
import { useNavigate, Link, useParams } from "react-router-dom";
import { toast } from "react-toastify";
import { getReceipt, createReceipt, updateReceipt } from "../../api/receipts.api.js";
import { getCustomer } from "../../api/customers.api.js";
import { formatBaht, formatDate } from "../../utils.js";
import InvoicePickerModal from "../InvoicePickerModal.jsx";
import CustomerPickerModal from "../components/CustomerPickerModal.jsx";
import { AlertModal } from "../components/Modal.jsx";
import Loading from "../components/Loading.jsx";

function today() {
  return new Date().toISOString().slice(0, 10);
}

export default function ReceiptPage({ mode: propMode }) {
  const { receiptNo } = useParams();
  const mode = propMode || (receiptNo ? "view" : "create");
  const nav = useNavigate();

  const [receiptData, setReceiptData] = React.useState(null);
  const [loading, setLoading] = React.useState(mode !== "create");
  const [err, setErr] = React.useState("");
  const [submitting, setSubmitting] = React.useState(false);
  const [alertModal, setAlertModal] = React.useState({ isOpen: false, message: "" });
  const [invoicePickerOpen, setInvoicePickerOpen] = React.useState(false);
  const [customerPickerOpen, setCustomerPickerOpen] = React.useState(false);

  // Form state
  const [receiptNoVal, setReceiptNoVal] = React.useState("");
  const [receiptNoAuto, setReceiptNoAuto] = React.useState(true);
  const [receiptDate, setReceiptDate] = React.useState(today());
  const [customerCode, setCustomerCode] = React.useState("");
  const [customerName, setCustomerName] = React.useState("");
  const [paymentMethod, setPaymentMethod] = React.useState("cash");

```

```

const [paymentNotes, setPaymentNotes] = React.useState("");
const [lineItems, setLineItems] = React.useState([]);

React.useEffect(() => {
  if (mode === "create") return;
  getReceipt(receiptNo)
    .then((data) => {
      setReceiptData(data);
      if (mode === "edit") {
        const h = data.header;
        setReceiptNoVal(h.receipt_no);
        setReceiptNoAuto(false);
        setReceiptDate(h.receipt_date ? h.receipt_date.slice(0, 10) : today());
        setCustomerCode(h.customer_code);
        setCustomerName(h.customer_name);
        setPaymentMethod(h.payment_method || "cash");
        setPaymentNotes(h.payment_notes || "");
        setLineItems(
          data.line_items.map((li) => ({
            invoice_no: li.invoice_no,
            amount_due: Number(li.amount_due),
            amount_already_received: Number(li.amount_already_received),
            amount_remain: Number(li.amount_due) - Number(li.amount_already_received),
            amount_received: Number(li.amount_received),
          })),
        );
      }
      setLoading(false);
    })
    .catch((e) => {
      setErr(String(e.message || e));
      setLoading(false);
    });
}, [receiptNo, mode]);

function handleCustomerCodeChange(newCode) {
  setCustomerCode(newCode);
  setCustomerName("");
  setLineItems([]);
}

async function handleCustomerBlur() {
  if (!customerCode) return;
  try {
    const cust = await getCustomer(customerCode);
    setCustomerName(cust.name || "");
  } catch {
    setCustomerName("");
  }
}

function handleCustomerSelected(code, label) {
  const name = label.startsWith(code + " - ") ? label.slice(code.length + 3) : label;
  setCustomerCode(code);
  setCustomerName(name);
  setLineItems([]);
}

function handleInvoiceSelected(inv) {
  if (lineItems.some((li) => li.invoice_no === inv.invoice_no)) {
    setInvoicePickerOpen(false);
    return;
  }
  setLineItems((prev) => [
    ...prev,
    {
      invoice_no: inv.invoice_no,
      amount_due: Number(inv.amount_due),
      amount_already_received: Number(inv.amount_already_received),
      amount_remain: Number(inv.amount_remain),
      amount_received: Number(inv.amount_remain),
    },
  ]);
  setInvoicePickerOpen(false);
}

function handleAmountChange(idx, val) {
  setLineItems((prev) =>
    prev.map((li, i) => (i === idx ? { ...li, amount_received: Number(val) || 0 } : li)),
  );
}

function handleRemoveLine(idx) {

```

```

    setLineItems((prev) => prev.filter((_, i) => i !== idx));
  }

  async function handleSubmit() {
    const errors = [];
    if (!receiptDate) errors.push("Receipt date is required.");
    if (!customerCode) errors.push("Customer is required.");
    if (lineItems.length === 0) errors.push("At least one invoice line is required.");
    lineItems.forEach((li, i) => {
      if (!li.invoice_no) errors.push(`Line ${i + 1}: Invoice No is required.`);
      if (!(li.amount_received > 0)) errors.push(`Line ${i + 1}: Amount received must be > 0.`);
    });
    const invoiceNos = lineItems.map((li) => li.invoice_no);
    if (new Set(invoiceNos).size !== invoiceNos.length) errors.push("Duplicate invoices in line items.");

    if (errors.length > 0) {
      setAlertModal({
        isOpen: true,
        message: <ul style={{ margin: 0, paddingLeft: 16 }}>{errors.map((e, i) => <li key={i}>{e}</li>)}</ul>,
      });
      return;
    }

    setSubmitting(true);
    setErr("");
    try {
      const payload = {
        receipt_no: receiptNoAuto ? "" : receiptNoVal,
        receipt_date: receiptDate,
        customer_code: customerCode,
        payment_method: paymentMethod,
        payment_notes: paymentNotes,
        line_items: lineItems.map((li) => ({ invoice_no: li.invoice_no, amount_received: li.amount_received })),
      };
      if (mode === "create") {
        const res = await createReceipt(payload);
        toast.success("Receipt created.");
        nav(`/receipts/${encodeURIComponent(res.receipt_no)}`);
      } else {
        await updateReceipt(receiptNo, payload);
        toast.success("Receipt updated.");
        nav(`/receipts/${encodeURIComponent(receiptNo)}`);
      }
    } catch (e) {
      const msg = String(e.message || e);
      setErr(msg);
      toast.error(msg);
    } finally {
      setSubmitting(false);
    }
  }

  if (loading) return <Loading size="large" />;

  // View mode
  if (mode === "view" && receiptData) {
    const h = receiptData.header;
    const lines = receiptData.line_items || [];
    return (
      <div className="invoice-preview">
        <div className="page-header no-print">
          <h3 className="page-title">Receipt {h.receipt_no}</h3>
          <div className="flex gap-4">
            <Link to="/receipts" className="btn btn-outline">← Back</Link>
            <Link to={` /receipts/${encodeURIComponent(receiptNo)}/edit`} className="btn btn-outline">Edit</Link>
            <button onClick={() => window.print()} className="btn btn-primary">
              <svg style={{ marginRight: 8 }} width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeW
                <path d="M6 9V2h12V7"></path>
                <path d="M6 18H4a2 2 0 0 1-2-2v-5a2 2 0 0 1 2-2h16a2 2 0 0 1 2 2v5a2 2 0 0 1-2 2h-2"></path>
                <rect x="6" y="14" width="12" height="8"></rect>
              </svg>
              Print PDF
            </button>
          </div>
        </div>
        <div className="card">
          <div className="flex justify-between mb-4">
            <div>
              <div className="brand mb-4">InvoiceDoc v2</div>
              <div className="font-bold">Customer</div>
              <div>{h.customer_name}</div>
              <div className="text-muted">{h.customer_code}</div>
            </div>

```

```

    </div>
    <div className="text-right">
      <h2 className="mb-4">RECEIPT</h2>
      <div><span className="font-bold">Date:</span> {formatDate(h.receipt_date)}</div>
      <div><span className="font-bold">Receipt No:</span> {h.receipt_no}</div>
      <div><span className="font-bold">Payment Method:</span> {h.payment_method}</div>
      {h.payment_notes} && <div><span className="font-bold">Notes:</span> {h.payment_notes}</div>
    </div>
  </div>
  <div className="table-container">
    <table className="modern-table">
      <thead>
        <tr>
          <th>Invoice No</th>
          <th className="text-right">Amount Due</th>
          <th className="text-right">Already Received</th>
          <th className="text-right">Balance Before</th>
          <th className="text-right">Amount Received Here</th>
          <th className="text-right">Balance After</th>
        </tr>
      </thead>
      <tbody>
        {lines.map((li) => {
          const balanceBefore = Number(li.amount_due) - Number(li.amount_already_received);
          const balanceAfter = balanceBefore - Number(li.amount_received);
          return (
            <tr key={li.id}>
              <td>{li.invoice_no}</td>
              <td className="text-right">{formatBaht(li.amount_due)}</td>
              <td className="text-right">{formatBaht(li.amount_already_received)}</td>
              <td className="text-right">{formatBaht(balanceBefore)}</td>
              <td className="text-right">{formatBaht(li.amount_received)}</td>
              <td className="text-right">{formatBaht(balanceAfter)}</td>
            </tr>
          );
        })}
      </tbody>
    </table>
  </div>
  <div className="mt-4 flex justify-end">
    <div style={{ minWidth: 240 }}>
      <div className="flex justify-between mt-4 p-2 bg-body font-bold" style={{ fontSize: "1.1rem" }}>
        <span>Total Received:</span>
        <span>{formatBaht(h.total_received)}</span>
      </div>
    </div>
  </div>
</div>
);
}

// Create/Edit mode
const totalReceived = lineItems.reduce((s, li) => s + Number(li.amount_received || 0), 0);
const title = mode === "create" ? "Create Receipt" : `Edit Receipt ${receiptNo}`;

return (
  <div className="invoice-page">
    <AlertModal
      isOpen={alertModal.isOpen}
      onClose={() => setAlertModal({ isOpen: false, message: "" })}
      title="Validation Error"
      message={alertModal.message}
    />
    <CustomerPickerModal
      isOpen={customerPickerOpen}
      onClose={() => setCustomerPickerOpen(false)}
      onSelect={handleCustomerSelected}
    />
    <InvoicePickerModal
      isOpen={invoicePickerOpen}
      onClose={() => setInvoicePickerOpen(false)}
      onSelect={handleInvoiceSelected}
      customerCode={customerCode}
      excludeReceiptNo={mode === "edit" ? receiptNo : null}
    />

    <div className="page-header">
      <h3 className="page-title">{title}</h3>
      <Link to="/receipts" className="btn btn-outline">
        <svg style={{ marginRight: 8 }} width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth=
          <line x1="19" y1="12" x2="5" y2="12"></line>

```

```

        <polyline points="12 19 5 12 12 5"></polyline>
    </svg>
    Back
</Link>
</div>

{err && <div className="alert alert-error">{err}</div>}

<div className="card">
  <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", gap: "1rem" }}>
    /* Receipt No */
    <div className="form-group">
      <label className="form-label">Receipt No</label>
      <div style={{ display: "flex", gap: 8, alignItems: "center" }}>
        <input
          type="text"
          className="form-control"
          value={receiptNoAuto ? "(Auto)" : receiptNoVal}
          onChange={(e) => setReceiptNoVal(e.target.value)}
          disabled={receiptNoAuto || mode === "edit"}
          placeholder="Auto-generated"
        />
        {mode === "create" && (
          <label style={{ display: "flex", alignItems: "center", gap: 4, whiteSpace: "nowrap" }}>
            <input
              type="checkbox"
              checked={receiptNoAuto}
              onChange={(e) => setReceiptNoAuto(e.target.checked)}
            />
            Auto
          </label>
        )}
      </div>
    </div>
  </div>

  /* Receipt Date */
  <div className="form-group">
    <label className="form-label">Receipt Date *</label>
    <input
      type="date"
      className="form-control"
      value={receiptDate}
      onChange={(e) => setReceiptDate(e.target.value)}
    />
  </div>

  /* Customer Code */
  <div className="form-group">
    <label className="form-label">Customer Code *</label>
    <div style={{ display: "flex", gap: 8 }}>
      <input
        type="text"
        className="form-control"
        value={customerCode}
        onChange={(e) => handleCustomerCodeChange(e.target.value)}
        onBlur={handleCustomerBlur}
        placeholder="e.g. C001"
      />
      <button
        type="button"
        className="btn btn-outline"
        onClick={() => setCustomerPickerOpen(true)}
        title="Select customer"
      >
        LoV
      </button>
      <button
        type="button"
        className="btn btn-outline"
        onClick={() => handleCustomerCodeChange("")}
        title="Clear customer"
      >
        x
      </button>
    </div>
  </div>

  /* Customer Name */
  <div className="form-group">
    <label className="form-label">Customer Name</label>
    <input type="text" className="form-control" value={customerName} readOnly />
  </div>

```

```

    { /* Payment Method */ }
    <div className="form-group">
      <label className="form-label">Payment Method</label>
      <select
        className="form-control"
        value={paymentMethod}
        onChange={(e) => setPaymentMethod(e.target.value)}
      >
        <option value="cash">Cash</option>
        <option value="bank transfer">Bank Transfer</option>
        <option value="check">Check</option>
      </select>
    </div>

    { /* Payment Notes */ }
    <div className="form-group">
      <label className="form-label">Payment Notes</label>
      <textarea
        className="form-control"
        value={paymentNotes}
        onChange={(e) => setPaymentNotes(e.target.value)}
        rows={2}
      />
    </div>
  </div>
</div>

{ /* Line Items */ }
<div className="card" style={{ marginTop: "1rem" }}>
  <div style={{ display: "flex", justifyContent: "space-between", alignItems: "center", marginBottom: "1rem" }}>
    <h4 style={{ margin: 0 }}>Invoice Lines</h4>
    <button
      type="button"
      className="btn btn-outline"
      onClick={() => {
        if (!customerCode) {
          showAlertModal({ isOpen: true, message: "Please enter a customer code first." });
          return;
        }
        setInvoicePickerOpen(true);
      }}
    >
      + Add Invoice
    </button>
  </div>
  <div className="table-container">
    <table className="modern-table">
      <thead>
        <tr>
          <th>Invoice No</th>
          <th className="text-right">Full Amount Due</th>
          <th className="text-right">Already Received</th>
          <th className="text-right">Amount Remaining</th>
          <th className="text-right">Amount Received Here</th>
          <th className="text-right">Still Remaining</th>
        </tr>
      </thead>
      <tbody>
        {lineItems.length === 0 && (
          <tr>
            <td colspan={7} style={{ textAlign: "center", padding: 24, color: "var(--text-muted)" }}>
              No invoice lines added. Click &ldquo;Add Invoice&rdquo; to begin.
            </td>
          </tr>
        )}
        {lineItems.map((li, idx) => {
          const stillRemaining = li.amount_remain - li.amount_received;
          return (
            <tr key={li.invoice_no}>
              <td>{li.invoice_no}</td>
              <td className="text-right">{formatBaht(li.amount_due)}</td>
              <td className="text-right">{formatBaht(li.amount_already_received)}</td>
              <td className="text-right">{formatBaht(li.amount_remain)}</td>
              <td className="text-right">
                <input
                  type="number"
                  className="form-control"
                  style={{ textAlign: "right", width: 130 }}
                  value={li.amount_received}
                  min={0}
                  step={0.01}
                />
              </td>
            </tr>
          )
        })}
      </tbody>
    </table>
  </div>

```

```

        onChange={(e) => handleAmountChange(idx, e.target.value)}
      />
    </td>
    <td className="text-right">{formatBaht(stillRemaining)}</td>
  </tr>
  <tr>
    <td>
      <button
        type="button"
        className="btn btn-outline"
        style={{ fontSize: "0.7rem", padding: "4px 8px", color: "#ef4444", borderColor: "#ef4444" }}
        onClick={() => handleRemoveLine(idx)}
      >
        Remove
      </button>
    </td>
  </tr>
</table>
</div>

/* Summary */
<div style={{ marginTop: "1rem", display: "flex", justifyContent: "flex-end" }}>
  <div style={{ minWidth: 260 }}>
    <div
      className="flex justify-between mt-2 p-2 font-bold"
      style={{ fontSize: "1.1rem", background: "var(--bg-body, #f8fafc)", borderRadius: "var(--radius)" }}
    >
      <span>Total Received:</span>
      <span>{formatBaht(totalReceived)}</span>
    </div>
    <div style={{ marginTop: "1rem", textAlign: "right" }}>
      <button
        type="button"
        className="btn btn-primary"
        onClick={handleSubmit}
        disabled={submitting}
      >
        {submitting ? "Saving..." : mode === "create" ? "Create Receipt" : "Update Receipt"}
      </button>
    </div>
  </div>
</div>
</div>
</div>
</div>
);
}

```

client/src/pages/reports/ReceiptReports.jsx

```

import React from "react";
import { fetchReceiptList, fetchInvoiceReceiptReport } from "../../../api/receiptReports.api.js";
import { formatBaht, formatDate } from "../../../utils.js";

const TABS = [
  { key: "receipt-list", label: "Receipt List" },
  { key: "invoice-receipt", label: "Invoice & Receipt" },
];

export default function ReceiptReports({ tab: initialTab }) {
  const [activeTab, setActiveTab] = React.useState(initialTab);
  const [filters, setFilters] = React.useState({ date_from: "", date_to: "", customer_code: "" });
  const [hasRun, setHasRun] = React.useState(false);
  const [data, setData] = React.useState([]);
  const [loading, setLoading] = React.useState(false);
  const [error, setError] = React.useState("");

  React.useEffect(() => {
    setActiveTab(initialTab);
    setHasRun(false);
    setData([]);
    setError("");
  }, [initialTab]);

  function handleFilterChange(e) {
    setFilters((f) => ({ ...f, [e.target.name]: e.target.value }));
  }

  async function runReport() {

```



```

setLoading(true);
setError("");
setHasRun(true);
try {
  let result;
  if (activeTab === "receipt-list") {
    result = await fetchReceiptList({ ...filters, limit: 1000 });
  } else {
    result = await fetchInvoiceReceiptReport({ ...filters, limit: 1000 });
  }
  setData(result.data || []);
} catch (e) {
  setError(String(e.message || e));
} finally {
  setLoading(false);
}
}

function resetReport() {
  setFilters({ date_from: "", date_to: "", customer_code: "" });
  setHasRun(false);
  setData([]);
  setError("");
}

const grandTotalReceived = activeTab === "receipt-list"
  ? data.reduce((s, r) => s + Number(r.total_received || 0), 0)
  : 0;

return (
  <div>
    <div className="page-header">
      <h3 className="page-title">Receipt Reports</h3>
    </div>

    {/* Tabs */}
    <div style={{ display: "flex", gap: 0, marginBottom: "1rem", borderBottom: "2px solid var(--border)" }}>
      {TABS.map((tab) => (
        <button
          key={tab.key}
          type="button"
          onClick={() => { setActiveTab(tab.key); setHasRun(false); setData([]); setError(""); }}
          style={{
            padding: "10px 24px",
            border: "none",
            background: "none",
            cursor: "pointer",
            fontWeight: activeTab === tab.key ? 700 : 400,
            color: activeTab === tab.key ? "var(--primary)" : "var(--text-muted)",
            borderBottom: activeTab === tab.key ? "2px solid var(--primary)" : "2px solid transparent",
            marginBottom: -2,
          }}
        >
          {tab.label}
        </button>
      ))}
    </div>

    {/* Filter Bar */}
    <div className="card" style={{ marginBottom: "1rem" }}>
      <div style={{ display: "flex", gap: "1rem", alignItems: "flex-end", flexWrap: "wrap" }}>
        <div className="form-group" style={{ margin: 0 }}>
          <label className="form-label">Date From</label>
          <input
            type="date"
            className="form-control"
            name="date_from"
            value={filters.date_from}
            onChange={handleFilterChange}
          />
        </div>
        <div className="form-group" style={{ margin: 0 }}>
          <label className="form-label">Date To</label>
          <input
            type="date"
            className="form-control"
            name="date_to"
            value={filters.date_to}
            onChange={handleFilterChange}
          />
        </div>
        <div className="form-group" style={{ margin: 0 }}>

```

```

<label className="form-label">Customer Code</label>
<input
  type="text"
  className="form-control"
  name="customer_code"
  value={filters.customer_code}
  onChange={handleFilterChange}
  placeholder="e.g. C001"
/>
</div>
<button type="button" className="btn btn-primary" onClick={runReport}>
  Run Report
</button>
<button type="button" className="btn btn-outline" onClick={resetReport}>
  Reset
</button>
</div>
</div>

{error && <div className="alert alert-error">{error}</div>}

{loading && (
  <div style={{ textAlign: "center", padding: 48, color: "var(--text-muted)" }}>
    Running report...
  </div>
)}

{hasRun && !loading && (
  <div className="card">
    {activeTab === "receipt-list" && (
      <div className="table-container">
        <table className="modern-table">
          <thead>
            <tr>
              <th>Receipt No</th>
              <th>Date</th>
              <th>Customer Code</th>
              <th>Customer Name</th>
              <th>Payment Method</th>
              <th>Notes</th>
              <th className="text-right">Total Received</th>
            </tr>
          </thead>
          <tbody>
            {data.length === 0 && (
              <tr>
                <td colspan={7} style={{ textAlign: "center", padding: 24, color: "var(--text-muted)" }}>
                  No data found.
                </td>
              </tr>
            )}
            {data.map((r) => (
              <tr key={r.receipt_no}>
                <td>{r.receipt_no}</td>
                <td>{formatDate(r.receipt_date)}</td>
                <td>{r.customer_code}</td>
                <td>{r.customer_name}</td>
                <td>{r.payment_method}</td>
                <td>{r.payment_notes || "-"}</td>
                <td className="text-right">{formatBaht(r.total_received)}</td>
              </tr>
            ))}
            {data.length > 0 && (
              <tr style={{ fontWeight: 700, background: "var(--bg-body, #f8f9fc)" }}>
                <td colspan={6} style={{ textAlign: "right" }}>Grand Total:</td>
                <td className="text-right">{formatBaht(grandTotalReceived)}</td>
              </tr>
            )}
          </tbody>
        </table>
      </div>
    )}
  </div>
)}

{activeTab === "invoice-receipt" && (
  <div className="table-container">
    <table className="modern-table">
      <thead>
        <tr>
          <th>Invoice No</th>
          <th>Invoice Date</th>
          <th>Customer Code</th>
          <th>Customer Name</th>

```

```

        <th className="text-right">Amount Due</th>
        <th className="text-right">Total Received</th>
        <th className="text-right">Remaining</th>
        <th>Receipt No</th>
        <th>Receipt Date</th>
        <th className="text-right">Receipt Amount</th>
    </tr>
</thead>
<tbody>
    {data.length === 0 && (
        <tr>
            <td colspan={10} style={{ textAlign: "center", padding: 24, color: "var(--text-muted)" }}>
                No data found.
            </td>
        </tr>
    )}
    {data.map((r, idx) => (
        <tr key={idx}>
            <td>{r.invoice_no}</td>
            <td>{formatDate(r.invoice_date)}</td>
            <td>{r.customer_code}</td>
            <td>{r.customer_name}</td>
            <td className="text-right">{formatBaht(r.amount_due)}</td>
            <td className="text-right">{formatBaht(r.total_received)}</td>
            <td
                className="text-right"
                style={{ color: Number(r.amount_remain) > 0 ? "#ef4444" : "#22c55e" }}
            >
                {formatBaht(r.amount_remain)}
            </td>
            <td>{r.receipt_no || "-"}</td>
            <td>{r.receipt_date ? formatDate(r.receipt_date) : "-"}</td>
            <td className="text-right">
                {r.receipt_amount !== null ? formatBaht(r.receipt_amount) : "-"}
            </td>
        </tr>
    ))}
</tbody>
</table>
</div>
    )}
</div>
    )}
</div>
    );
}

```